

Dataset Praktikum Language Model Mini Berbasis Transformer

Topik: Deep Learning, CNN, RNN, Transformer, dan Large Language Model

1. Deep learning adalah cabang dari machine learning yang menggunakan jaringan saraf tiruan dengan banyak lapisan. Model deep learning belajar dari data melalui proses training, yaitu proses memperbarui bobot berdasarkan kesalahan prediksi.
2. Neural network terdiri dari neuron, bobot, bias, fungsi aktivasi, dan fungsi loss. Input diproses melalui beberapa lapisan hingga menghasilkan output. Jika output belum sesuai target, model melakukan backpropagation untuk memperbaiki bobot.
3. Convolutional Neural Network atau CNN sering digunakan untuk pengolahan citra. CNN memanfaatkan filter atau kernel untuk mengenali pola visual seperti garis, tepi, tekstur, bentuk, dan objek pada gambar.
4. Pooling pada CNN digunakan untuk mengurangi ukuran fitur tanpa menghilangkan informasi penting. Dengan pooling, proses komputasi menjadi lebih ringan dan model lebih tahan terhadap perubahan posisi objek kecil.
5. Recurrent Neural Network atau RNN digunakan untuk data berurutan seperti teks, suara, dan deret waktu. RNN membaca data secara berurutan dan menyimpan informasi sebelumnya dalam hidden state.
6. RNN memiliki kelemahan saat menangani urutan yang sangat panjang. Informasi lama dapat hilang atau sulit dipertahankan. Masalah ini dikenal sebagai vanishing gradient dan exploding gradient.
7. LSTM dan GRU dikembangkan untuk mengatasi kelemahan RNN. Keduanya menggunakan mekanisme gerbang untuk mengatur informasi mana yang disimpan, dilupakan, atau diteruskan ke langkah berikutnya.
8. Transformer adalah arsitektur deep learning yang menggunakan mekanisme attention. Transformer tidak membaca teks secara satu per satu seperti RNN, tetapi dapat memperhatikan hubungan antar token dalam sebuah urutan.
9. Self-attention membuat model dapat menghitung hubungan antara satu token dengan token lain dalam kalimat. Misalnya, pada kalimat panjang, model dapat melihat kata mana yang paling relevan untuk memahami konteks.
10. Multi-head attention menggunakan beberapa kepala attention secara bersamaan. Setiap kepala dapat belajar pola hubungan yang berbeda, sehingga representasi teks menjadi lebih kaya.
11. Positional embedding diperlukan karena Transformer tidak memiliki urutan alami seperti RNN. Positional embedding memberi informasi posisi token agar model mengetahui urutan teks.
12. Language model adalah model yang mempelajari pola bahasa. Salah satu tugas utama language model adalah next-token prediction, yaitu memprediksi token berikutnya berdasarkan token-token sebelumnya.
13. Large Language Model atau LLM adalah model bahasa berukuran besar yang dilatih menggunakan data sangat banyak. LLM modern banyak menggunakan arsitektur Transformer karena efektif memodelkan hubungan dalam teks.
14. Tokenizer mengubah teks menjadi token. Token dapat berupa karakter, subword, kata, atau simbol tertentu. Model tidak membaca teks mentah, tetapi membaca angka hasil tokenisasi.

- 15.** Embedding mengubah token yang berbentuk angka menjadi vektor. Vektor ini berisi representasi numerik yang dapat diproses oleh neural network.
- 16.** Causal mask digunakan pada model bahasa generatif agar model tidak melihat token masa depan saat training. Model hanya boleh memprediksi token berikutnya berdasarkan token sebelumnya.
- 17.** Loss function mengukur seberapa jauh prediksi model dari target. Dalam language model, cross entropy sering digunakan untuk mengukur kesalahan prediksi token berikutnya.
- 18.** Optimizer seperti AdamW digunakan untuk memperbarui bobot model selama training. Tujuan training adalah menurunkan nilai loss secara bertahap.
- 19.** Evaluasi model bahasa dapat dilakukan dengan melihat validation loss, perplexity, dan hasil teks yang dihasilkan. Perplexity yang lebih rendah biasanya menunjukkan model lebih baik dalam memprediksi token.
- 20.** Model kecil pada praktikum ini belum dapat menjawab seperti chatbot besar. Namun model ini menunjukkan prinsip dasar LLM, yaitu tokenisasi, embedding, attention, training, evaluasi, dan generasi teks.
- 21.** Setelah model dilatih di Google Colab, file model dapat disimpan dan diunduh. Model tersebut kemudian dapat digunakan secara lokal di Visual Studio Code melalui aplikasi Streamlit.
- 22.** Streamlit memungkinkan program Python sederhana diubah menjadi aplikasi web interaktif. Dengan Streamlit, pengguna dapat memasukkan prompt dan melihat teks yang dihasilkan oleh model.
- 23.** Mini language model dalam praktikum ini menggunakan token karakter. Token karakter lebih mudah dipahami karena setiap huruf, spasi, dan tanda baca menjadi token tersendiri.
- 24.** Pada model karakter, vocabulary berisi semua karakter unik yang muncul pada dataset. Jika dataset berisi huruf kecil, huruf besar, angka, spasi, dan tanda baca, semua karakter tersebut menjadi bagian vocabulary.
- 25.** Semakin besar dataset dan semakin lama training, model biasanya dapat menghasilkan teks yang lebih konsisten. Namun model yang terlalu kecil tetap memiliki batas kemampuan.
- 26.** Hyperparameter seperti jumlah embedding, jumlah attention head, jumlah layer, block size, batch size, learning rate, dan jumlah iterasi memengaruhi hasil training.
- 27.** Model generatif menghasilkan teks dengan mengambil sampel token berikutnya dari distribusi probabilitas. Temperature dapat digunakan untuk mengatur tingkat variasi hasil generasi.
- 28.** Jika temperature terlalu rendah, hasil model cenderung aman dan berulang. Jika temperature terlalu tinggi, hasil model lebih bervariasi tetapi sering tidak stabil.
- 29.** Project ini memperlihatkan hubungan antara teori deep learning dan aplikasi modern berbasis LLM. Mahasiswa dapat melihat bahwa LLM dibangun dari konsep dasar neural network, embedding, attention, loss, dan optimizer.
- 30.** Pemahaman terhadap mini language model membantu mahasiswa memahami batasan dan kekuatan LLM. LLM bukan sekadar aplikasi chatbot, melainkan model probabilistik yang mempelajari pola token dari data.